MLOPS · FORMATION 2 × 4H

CI/CD appliqué au Machine Learning

Pipelines, automatisation, monitoring et gouvernance sur Azure

AZURE ML | GITHUB ACTIONS | AZURE PIPELINES

Ce que vous allez maîtriser

OBJECTIFS D'APPRENTISSAGE

- Expliquer les **spécificités d'un pipeline CI/CD ML** par rapport au logiciel classique
- Lire et écrire des **pipelines YAML** pour GitHub Actions et Azure Pipelines
- Mettre en place un **quality gate** automatisé sur les métriques du modèle
- Choisir entre **AKS** et **AML Managed Endpoint** selon le contexte
- Configurer le **monitoring applicatif et ML** avec App Insights et KQL
- Simuler et analyser un **drift de données** en production
- Appliquer les bonnes pratiques de **gouvernance** : RBAC, Key Vault, OIDC

PLAN DE LA FORMATION

5 parties · 8h au total

MATIN · 4H

Fondations CI/CD ML

Partie 1 · Fondations ML
Partie 2 · Infra & Environnements
Partie 3 · CI/CD & Outils
— Démo guidée 1 —

APRÈS-MIDI · 4H

Serving, Monitoring & Gouvernance

Partie 3 · Serving (AKS vs AML)
Partie 4 · Tracking, Registry & Drift
— Démo guidée 2 —
Partie 5 · Gouvernance

La journée suit le même rythme : **théorie + quiz interactifs** → **démo guidée** → **récap collectif**

Pour cette formation en **une journée**, l'environnement `dev` de démonstration est supposé **préparé en amont** — cela condense les étapes de setup et d'infrastructure détaillées dans le lab complet.

PARTIE 1 — FONDATIONS

Pourquoi le CI/CD change avec le ML ?

Du notebook au système industrialisé

Un notebook qui marche… et après ?

Situation typique

- Notebook qui tourne sur **ma machine**
- Données en local ou dans un Drive
- Évaluation faite **une fois** à la main
- Déploiement : "j'ai envoyé le pkl par email"
- Monitoring : "on verra si ça plante"

Ce qu'on vise

- Scripts **relancables** sur n'importe quel compute
- Entraînement **tracé et versionné**
- Quality gate **automatique**
- Déploiement **reproductible** via pipeline
- Monitoring **continu** après mise en production

Le MLOps commence quand on accepte que ces cinq points ont **autant d'importance que le score du modèle**.

LES 4 QUESTIONS MLOPS

Un projet ML exploitable répond à 4 questions

1

Comment entraîner et évaluer proprement ?

2

Comment mettre en service sans bricolage manuel ?

3

Comment observer ce qui se passe après le déploiement ?

4

Comment contrôler qui a le droit de faire quoi ?

Le CI/CD ML est la réponse concrète aux questions **1** et **2**. Le monitoring répond à la **3**. La gouvernance à la **4**.

QUIZ · VÉRIFICATION RAPIDE

CI ou CD ?

QUESTION 1 / 4

Un pipeline exécute automatiquement `black`, `flake8` et `pytest` à chaque push. C'est de la…

- A · Intégration Continue (CI)
- B · Déploiement Continu (CD)
- C · Les deux

Réponse : A — La CI vérifie que le changement est acceptable. La CD livre l'artefact dans un environnement.

PARTIE 1 · SUITE

Spécificités des pipelines ML

Ce qui change par rapport au logiciel classique

ML VS LOGICIEL CLASSIQUE

Ce qui change avec le ML

	PROJET LOGICIEL CLASSIQUE	PROJET ML
Ce qu'on vérifie	Qualité du code + tests	Code **+** pipeline d'entraînement **+** métriques
Ce qu'on livre	Un binaire ou une image	Un comportement de **prédiction**
Quality gate	Tests unitaires / intégration	Accuracy · F1 · seuil configurable
Artefacts	`.jar` · `.whl` · image Docker	Modèle + scaler + métadonnées MLflow
Durée de la CI	Secondes à quelques min	**Plusieurs minutes** (job AML inclus)

Une CI ML qui valide seulement le code local ne valide **pas encore** que l'entraînement tourne dans Azure ML.

3 étapes · 1 quality gate

```yaml
# mlops/pipelines/pipeline.yml (extrait)
jobs:
  prep_data:
    component: azureml:prep_data@latest
  train_model:
    component: azureml:train_model@latest
    inputs: { training_data: ${{parent.jobs.prep_data.outputs.training_data}} }
  evaluate_model:
    component: azureml:evaluate_model@latest
    inputs: { model: ${{parent.jobs.train_model.outputs.model}} }
```

Ce que `prep.py` vérifie avant d'entraîner

Structure attendue

- 4 features numériques (cm)
- 1 colonne `target` (0, 1, 2)
- Aucune valeur nulle
- 150 lignes minimum

Ce que fait `prep.py` dans ce repo

- Charge Iris depuis sklearn
- Renomme les colonnes
- Split 80/20 stratifié
- Sauvegarde `train.csv` + `test.csv`

En production : vérifier aussi

- Plage de valeurs (ex: sepal ∈ [4,8])
- Distribution des classes (déséquilibre ?)
- Schéma : types de colonnes corrects
- **Bloquer le pipeline si KO**

Dans ce lab, les données viennent de sklearn → toujours propres. En vrai projet, **la validation des données est le premier quality gate**, avant même l'entraînement.

Le pipeline peut — et doit — échouer

```python
# evaluate_model/evaluate.py (extrait)
accuracy = accuracy_score(y_test, y_pred)
mlflow.log_metric("accuracy", accuracy)

THRESHOLD = float(os.environ.get(
    "ACCURACY_THRESHOLD", "0.90"
))

if accuracy < THRESHOLD:
    print(f"FAIL: {accuracy:.3f} < {THRESHOLD}")
    sys.exit(1)          # fait échouer le job AML
else:
    print(f"PASS: {accuracy:.3f} >= {THRESHOLD}")
```

Si accuracy ≥ 0.90

Job AML → succès

CI GitHub → verte

CD dev peut être déclenché

Si accuracy < 0.90

`sys.exit(1)` → job AML rouge

CI GitHub bloquée

CD **non déclenché**

À vous de jouer

EXERCICE · 8 MIN · EN BINÔME

Ouvrez `.github/workflows/ci-train.yml` dans le repo.

Répondez aux 3 questions :

1. Sur quel événement Git ce workflow se déclenche-t-il ?
2. Quelle étape soumet le pipeline à Azure ML ? Quel script est appelé ?
3. Que se passerait-il si on changeait le seuil `ACCURACY_THRESHOLD` à `0.99` ?

Repère clé : la commande `az ml job create` + le flag `--stream` qui bloque GitHub Actions jusqu'à la fin du job AML.

PARTIE 2 — INFRASTRUCTURE

IaC & Environnements

Terraform crée la plateforme, le CI/CD l'exploite

CE QUE TERRAFORM PROVISIONNE

Un environnement complet en une commande

Compute & Serving

- `azurerm_machine_learning_workspace`
- `azurerm_kubernetes_cluster` (AKS)
- `azurerm_container_registry` (ACR)

Observabilité & Secrets

- `azurerm_application_insights`
- `azurerm_log_analytics_workspace`
- `azurerm_key_vault`

Automatisations RBAC

- `AcrPull` pour AKS → ACR
- `AcrPull` pour AML → ACR
- Séparation `dev` / `prod` par variable

Un même `main.tf` déploie `dev` et `prod` — seul le fichier `.tfvars` change. Le backend remote stocke l'état dans un Storage Account Azure.

POURQUOI C'EST IMPORTANT POUR LE CI/CD

L'infra conditionne tout le reste

Sans IaC

- Ressources créées manuellement au portail
- Différences non tracées entre dev et prod
- Pas de revue de code sur l'infra
- Impossible de recréer l'environnement

Avec IaC (Terraform)

- `terraform plan` montre les changements avant
- Même code → même infra dev & prod
- PR review sur chaque changement d'infra
- Rollback possible via l'historique Git

Pour cette formation, l'environnement `dev` est **déjà provisionné**. Le code Terraform est dans `infrastructure/terraform/` — consultable à tout moment.

PARTIE 3 — CI/CD

Outils CI/CD :

GitHub Actions & Azure Pipelines

Deux syntaxes, un même modèle d'exploitation

Anatomie d'un workflow

```yaml
# .github/workflows/ci-train.yml
name: CI — Train & Evaluate

on:
  push:
    branches: [dev]          # déclenché sur push → dev
  pull_request:
    branches: [main]         # vérifié sur PR → main

permissions:
  id-token: write            # nécessaire pour OIDC
  contents: read

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Lint
        run: black --check . && flake8 .
      - name: Tests
        run: pytest tests/ -v

  train:
    needs: lint-and-test       # dépendance explicite
    steps:
      - uses: azure/login@v2   # authentification OIDC
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```

ÉQUIVALENCE AZURE DEVOPS

Même logique, autre vocabulaire

`workflow`	`pipeline`	unité d'automatisation
`on: push`	`trigger`	déclencheur
`jobs`	`stages / jobs`	unités d'exécution
`steps`	`steps`	actions individuelles
`uses: azure/login`	`AzureCLI@2`	auth Azure
`secrets.AZURE_*`	`Variable Groups`	secrets
GitHub Environments	Approvals & Gates	promotion
`needs`	`dependsOn`	dépendances

Le modèle d'exploitation est identique : événement Git → jobs ordonnés → artefacts → promotion contrôlée vers prod.

Pourquoi OIDC plutôt qu'un secret statique ?

Ancienne pratique

- Service Principal avec mot de passe
- Secret stocké dans GitHub Secrets
- Valide **indéfiniment** (jusqu'à rotation manuelle)
- Si leaké → accès Azure complet

Avec OIDC (federated credentials)

- GitHub demande un token temporaire à Entra ID
- Token valide **~1h** · non réutilisable
- **Aucun secret longue durée** dans GitHub
- Contexte borné : branche ou environment

```
# Dans le workflow — pas de mot de passe
- uses: azure/login@v2
  with:
```

Qu'est-ce qu'on stocke dans GitHub Secrets avec OIDC ?

QUESTION 2 / 4

Avec OIDC configuré, quels éléments se trouvent dans les GitHub Secrets ?

A · Le mot de passe du Service Principal

B · Le Client ID, Tenant ID et Subscription ID (pas de mot de passe)

C · Rien du tout, les secrets sont dans Key Vault

Réponse : B — Les IDs sont des identifiants, pas des secrets. C'est Azure Entra qui émet le token temporaire à la demande.

ARCHITECTURE DU LAB

Ce que vous allez manipuler

WORKFLOWS GITHUB

- `ci-train.yml` — lint + test + AML
- `cd-deploy-dev.yml` — image + AKS dev
- `cd-deploy-aml-endpoint.yml` — Managed Endpoint
- `cd-deploy-prod.yml` — promotion prod

AZURE RESOURCES

- Azure ML Workspace → jobs, registry
- Azure Container Registry → images
- AKS dev/prod → serving
- App Insights → telemetry HTTP
- Key Vault → secrets

STRATÉGIE DE BRANCHES

- `dev` → push déclenche CI auto
- `main` → PR et promotion prod
- CD dev → auto après CI verte, **rejouable manuellement**
- CD prod → approbation requise

TAKEAWAY · SÉCURITÉ CI/CD

À retenir avant la démo

POINTS CLÉS · PARTIES 1-3

- La CI ML valide le **code ET l'exécution dans Azure ML**
- Le quality gate sur les métriques **bloque la CD si le modèle est insuffisant**
- OIDC = **aucun secret longue durée** dans GitHub
- **L'identité du pipeline** compte autant que le code du pipeline
- Dans ce repo, le CD dev peut partir **automatiquement** après la CI, mais il est aussi **rejouable manuellement** pour la démonstration

DÉMO GUIDÉE · PARTIES 1-3

On passe à la démo

Exécution du pipeline CI/CD complet — en direct

DÉMO 1 · VUE D'ENSEMBLE

Ce qu'on va observer ensemble

1️⃣	2️⃣	3️⃣	4️⃣
Push → CI	**Pipeline AML**	**CD → AKS**	**Test live**
lint · tests unitaires · soumission pipeline AML	prep → train → evaluate → quality gate	image ACR · déploiement pod · endpoint HTTP	curl /predict → JSON · logs dans App Insights

📍 Repo : `.github/workflows/ci-train.yml` + `cd-deploy-dev.yml` + `mlops/pipelines/pipeline.yml`

Vérifier les prérequis RBAC

```
az account show \
  --query "{name:name, id:id}" -o table

az role assignment list \
  --resource-group rg-mlopslab-dev \
  --output table

az ml workspace show \
  --name aml-mlopslab-dev \
  --resource-group rg-mlopslab-dev \
  --query "{name:name,location:location}" \
  -o table
```

1. **Compte Azure actif**
   abonnement + tenant visible
2. **Role assignments OK**
   Contributor sur rg-mlopslab-dev
3. **Workspace AML accessible**
   aml-mlopslab-dev visible

Lire les workflows avant de pousser

```
# Afficher les triggers et jobs de la CI
grep -E "^on:|branches:|jobs:|needs:" \
  .github/workflows/ci-train.yml

# Afficher la commande de soumission AML
grep -A2 "az ml job create" \
  .github/workflows/ci-train.yml
```

Ce qu'on cherche à confirmer :

- Sur quelle branche la CI se déclenche
- Quelle étape dépend de quelle autre (`needs:`)
- Comment le résultat AML est attendu (`--stream`)

DÉMO 1 · ÉTAPE 3

Déclencher la CI avec un push

1. **Créer une branche feature depuis `dev`**

```
git checkout dev && git checkout -b feature/demo-cicd
```

2. **Faire un changement traçable**

```
echo "# demo $(date)" >> mlops/data-science/src/train.py
```

3. **Committer, pousser, ouvrir une PR vers `dev`**

```
git add . && git commit -m "demo" && git push origin feature/demo-cicd
```

4. **Observer le déclenchement dans GitHub Actions**

Onglet **Actions** → workflow `CI — Lint + Tests + AML Training Pipeline`

DÉMO 1 · ÉTAPE 4

Ce qu'on voit à l'écran

GitHub Actions

- Jobs : `lint` → `unit-tests` → `train-pipeline`
- Logs du job : `az ml job create`
- Status poll toutes les 30s
- ✅ vert si quality gate passé
- ❌ rouge si accuracy < 0.90

AML Studio (ml.azure.com)

- **Jobs** → run en cours
- 3 étapes : `prep_data` → `train_model` → `evaluate_model`
- **Metrics** : `train_accuracy` + `test_accuracy` + `test_f1_weighted`
- Comparaison de runs sur graphe

📍 **Point clé à montrer :** le lien entre le run GitHub Actions et le job AML — c'est la même exécution, vue des deux côtés.

Observer ou relancer le CD dev

Après une CI verte, le repo peut lancer le CD dev automatiquement via `workflow_run`.

Pour la démo, on peut aussi le relancer manuellement afin de rejouer les logs :

```
# Via GitHub UI : Actions → "CD — Deploy to Dev (ACR build + AKS deploy)" → Run workflow
# Ou via CLI :
gh workflow run cd-deploy-dev.yml \
  --ref dev
```

Ce que fait le workflow CD dev :

1. Reconstruit le modèle localement
2. Construit l'image Docker de serving
3. Pousse l'image dans ACR

Tester le service AKS

```
# Récupérer l'IP externe du service
kubectl get service iris-scoring-service \
  --namespace default \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}'

# Stocker l'IP
SCORING_URL="http://<IP_EXTERNE>/predict"

# Envoyer une prédiction normale
curl -X POST "$SCORING_URL" \
  -H "Content-Type: application/json" \
  -d '{"features": [5.1, 3.5, 1.4, 0.2]}'

# Résultat attendu
# {"prediction": 0, "class": "setosa"}
```

Si vous obtenez `connection refused` → vérifier que le pod AKS est en état `Running` avec `kubectl get pods`.

RÉCAP DÉMO 1

Ce qu'on vient de voir

CI

- Workflow CI déclenché sur push ✓
- Job AML exécuté (3 étapes) ✓
- Accuracy loggée dans MLflow ✓
- Quality gate passé (≥ 0.90) ✓

CD DEV

- CD déclenché manuellement ✓
- Image dans ACR ✓
- Pod AKS en état Running ✓
- curl /predict → réponse JSON ✓

CONCEPTS

- CI vs CD : différence claire ✓
- Quality gate : rôle compris ✓
- OIDC : pas de secret SP ✓
- CI locale ≠ CI cloud ✓

☕ Pause

Reprise dans 15 minutes

PARTIES 3-5 : SERVING · MONITORING · GOUVERNANCE

PARTIES 3-5 · APRÈS-MIDI

Serving, Monitoring & Gouvernance

Du déploiement à l'exploitation en production

PARTIE 3 · SERVING

PARTIE 4 · TRACKING

PARTIE 5 · GOUVERNANCE

RÉCAP · PARTIES 1-3

Ce qu'on a vu ce matin

Partie 1 · Fondations

La CI ML valide code + pipeline AML. La CD livre un comportement de prédiction, pas juste du code.

Partie 2 · Infra

Terraform provisionne l'environnement complet. Un même `main.tf` pour dev et prod.

Partie 3 · CI/CD

OIDC, quality gate, GH Actions + Démo : CI → CD dev → AKS → curl /predict ✓

PARTIE 3 · SERVING

Deux modes de serving

AKS ou AML Managed Endpoint : comment choisir ?

Serving sur AKS — Contrôle applicatif

Ce que l'équipe gère explicitement

- **Image Docker** avec runtime de serving dédié
- **Dépendances serving séparées** du training
- **Manifest Kubernetes** : Deployment + Service
- **IP externe** exposée par le LoadBalancer
- **Instrumentation Flask** vers App Insights

AKS = plus de contrôle sur le runtime, le réseau et le cycle de vie du conteneur.

Bonne pratique : séparer les dépendances training et serving. L'image AKS est plus légère et plus stable.

Serving géré — La logique plateforme

Ce qu'Azure ML prend en charge

- **Cycle endpoint / deployment**
- **Intégration native** avec le registre AML
- **Scaling** et runtime partiellement gérés
- Mais il faut toujours fournir `score.py`
- Et un **environnement d'inférence correct**

Managed Endpoint = plus rapide à industrialiser quand on veut rester proche des primitives AML.

Managed Endpoint ≠ "aucune question technique". Il faut encore un environnement d'inférence et un `score.py` compatible.

Cadre de décision

QUESTION	AKS	AML MANAGED ENDPOINT
Mon équipe a une culture Kubernetes	✅ Oui	Pas nécessaire
Je veux contrôler finement le runtime	✅ Oui	Moins
Je veux aller plus vite au départ	Moins	✅ Oui
Je veux un registre de modèles intégré	❌ Non automatique	✅ Oui
Je veux des logs HTTP dans App Insights	✅ Oui (Flask)	Possible
Je veux mutualiser avec d'autres workloads	✅ Oui	Non

Dans la séquence **versioning + monitoring** de l'après-midi, le modèle visible dans AML dépend du workflow **Managed Endpoint**, pas du déploiement AKS.

Quel workflow enregistre le modèle dans AML ?

QUESTION 3 / 4

Je veux que le modèle `iris-classifier` apparaisse dans le registre du workspace AML. Quel workflow déclencher ?

A · `ci-train.yml`

B · `cd-deploy-dev.yml` (déploiement AKS)

C · `cd-deploy-aml-endpoint.yml`

Réponse : C — Le déploiement AKS n'enregistre pas le modèle dans AML. Seul le workflow Managed Endpoint le fait.

PARTIE 4 — TRACKING & MONITORING

Monitoring multi-niveaux

Observer un modèle en production ne se résume pas à regarder la latence

Ce qu'on regarde à chaque niveau

NIVEAU	CE QU'ON REGARDE	OUTIL DANS CE REPO
Applicatif	Requêtes HTTP · erreurs · latence	Application Insights
ML	Runs · métriques · versions modèle	Azure ML Studio
Plateforme	Pods · rollout · service exposé	AKS / `kubectl`
Gouvernance	Qui a le droit d'agir	RBAC · Entra ID · Key Vault

Erreurs classiques :

· Une requête `200` ne dit rien sur la qualité du modèle

· Un bon score offline ne dit rien sur la santé du service

· Une alerte infra ne dit rien sur un drift métier

Observer le trafic HTTP du service AKS

```
// Requêtes récentes
requests
| where timestamp > ago(30m)
| order by timestamp desc
| take 20

// Succès / codes HTTP
requests
| where timestamp > ago(30m)
| summarize count() by success, resultCode

// Volume dans le temps (buckets 5 min)
requests
| where timestamp > ago(30m)
| summarize count() by bin(timestamp, 5m), success
| order by timestamp desc

// Traces applicatives
```

À vous d'écrire une requête

EXERCICE · 6 MIN

Dans Azure Portal → votre ressource App Insights → **Logs** (Query Hub).

Écrivez une requête KQL qui :

1. Prend les requêtes des **dernières 1h**
2. Filtre uniquement les requêtes en **échec** (`success == false`)
3. Affiche le `resultCode` , le `url` et le `timestamp`

Si aucune requête en échec : simuler une erreur avec `curl -X POST .../predict -d 'invalid'` puis relancer.

PARTIE 4 · VERSIONING

Registre de modèles & versioning

Du run MLflow au modèle en production

Tracking manuel avec mlflow.start_run()

```python
# mlops/data-science/src/train.py (extrait)
import mlflow
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

with mlflow.start_run():
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(max_iter=args.max_iter)),
    ])
    model.fit(X, y)
    acc = accuracy_score(y, model.predict(X))
    mlflow.log_param("max_iter", args.max_iter)
    mlflow.log_metric("train_accuracy", acc)
```

CYCLE DE VIE DU MODÈLE

Du run au modèle enregistré

🧪

Expérimentation

Runs MLflow dans AML Studio

→

📦

Enregistrement

`az ml model create` ou via Managed Endpoint workflow

→

🚀

Déploiement

Online endpoint pointant vers une version

→

🗄️

Archivage

Versions précédentes accessibles pour rollback

```
# Lister les versions d'un modèle enregistré
az ml model list --name iris-classifier \
  --workspace-name aml-mlopslab-dev \
  --resource-group rg-mlopslab-dev \
  --output table
```

Que retourne az ml model list ?

QUESTION 4 / 4

Vous lancez `az ml model list --name iris-classifier` et n'obtenez aucun résultat. Pourquoi ?

A · Le modèle n'a jamais été entraîné

B · Le workflow `cd-deploy-aml-endpoint.yml` n'a pas encore été exécuté

C · Le déploiement AKS n'a pas réussi

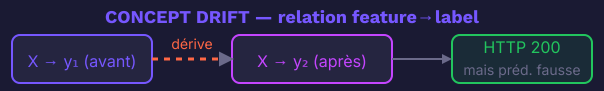
Réponse : B — Le déploiement AKS ne crée pas d'entrée dans le registre AML. Seul le workflow Managed Endpoint enregistre `iris-classifier`.

PARTIE 4 · DRIFT

Drift de données & alerting

Détecter la dégradation silencieuse d'un modèle

Deux types de dégradation à distinguer



Data Drift

La **distribution des entrées change**

Exemple : le capteur mesure maintenant en cm au lieu de mm → les features ont une nouvelle échelle

Le modèle reçoit des valeurs hors de sa zone d'entraînement

Concept Drift

La **relation entre features et label change**

Exemple : les habitudes d'achat changent après une crise → le modèle de recommandation prédit avec des patterns obsolètes

Le modèle répond, mais de moins en moins bien

LE PIÈGE DU 200 OK

Drift ≠ erreur HTTP

⚠️ **Point de confusion fréquent**

Le script de drift envoie des données hors distribution. L'API AKS peut continuer à répondre en 200 OK.

Donc : HTTP OK ≠ modèle fiable.

Ce qu'il faut vraiment suivre

- Distribution des features en entrée
- Distribution des prédictions en sortie
- Accuracy réelle si les labels sont disponibles

Alerte HTTP sur les requêtes en échec

1. Récupérer l'ID App Insights

```
AI_ID=$(az monitor app-insights component show \
  --app appi-mlopslab-dev \
  --resource-group rg-mlopslab-dev \
  --query id -o tsv)
```

2. Créer l'alerte métrique

```
az monitor metrics alert create \
  --name alert-failed-requests-dev \
  --resource-group rg-mlopslab-dev \
  --scopes "$AI_ID" \
  --condition "count requests/failed > 5" \
  --window-size 5m \
  --evaluation-frequency 1m
```

Un drift métier **n'est pas détectable par cette alerte**. Elle détecte les erreurs techniques, pas la dégradation des prédictions.

DÉMO GUIDÉE · PARTIES 3-5

On passe à la démo

Managed Endpoint · Versioning · Drift · RBAC

DÉMO 2 · VUE D'ENSEMBLE

Ce qu'on va observer ensemble

1️⃣	2️⃣	3️⃣	4️⃣
Managed Endpoint	**Model Registry**	**Alerting + Drift**	**RBAC + KV**
déploiement AML · enregistrement modèle	versioning · lien run MLflow · rollback	alerte Azure Monitor · simulation drift	moindre privilège · secrets sécurisés

📍 Repo : `cd-deploy-aml-endpoint.yml` · `scripts/generate-drift.py` · `scripts/setup-rbac.sh`

Déployer le Managed Endpoint AML

```
# Via GitHub UI : Actions → "CD — Deploy AML Managed Endpoint"
# → Run workflow → branch dev
gh workflow run cd-deploy-aml-endpoint.yml --ref dev

# Suivre le déploiement
gh run list --workflow=cd-deploy-aml-endpoint.yml \
  --limit 1

# Vérifier l'endpoint créé
az ml online-endpoint show \
  --name iris-endpoint-dev \
  --workspace-name aml-mlopslab-dev \
  --resource-group rg-mlopslab-dev \
  --query "{name:name, scoring_uri:scoring_uri, state:provisioning_state}" \
  -o table
```

Vérifier le modèle dans le registre

```
# Lister les versions du modèle enregistré
az ml model list \
  --name iris-classifier \
  --workspace-name aml-mlopslab-dev \
  --resource-group rg-mlopslab-dev \
  --output table

# Voir les détails d'une version spécifique
az ml model show \
  --name iris-classifier \
  --version 1 \
  --workspace-name aml-mlopslab-dev \
  --resource-group rg-mlopslab-dev \
  --query "{name:name, version:version, run_id:run_id}" \
  -o table
```

Configurer l'alerte Azure Monitor

```
# Récupérer l'ID App Insights
AI_ID=$(az monitor app-insights component show \
  --app appi-mlopslab-dev \
  --resource-group rg-mlopslab-dev \
  --query id -o tsv)

# Créer l'alerte
az monitor metrics alert create \
  --name "alert-failed-requests-dev" \
  --resource-group rg-mlopslab-dev \
  --scopes "$AI_ID" \
  --condition "count requests/failed > 5" \
  --window-size 5m \
  --evaluation-frequency 1m \
  --description "Failed requests > 5 en 5 min"

# Vérifier la création
az monitor metrics alert show \
  --name "alert-failed-requests-dev" \
  --resource-group rg-mlopslab-dev \
```

Simuler du trafic normal

```
# Stocker l'URL de scoring AKS
SCORING_URL="http://<IP_AKS>/predict"

# Envoyer 10 prédictions normales (fleur iris réelle)
for i in {1..10}; do
  curl -s -X POST "$SCORING_URL" \
    -H "Content-Type: application/json" \
    -d '{"features": [5.1, 3.5, 1.4, 0.2]}' \
  | python3 -c "import sys,json; r=json.load(sys.stdin); print(f'Pred: {r[\"class\"]}')"
done
```

Résultat attendu :

```
Pred: setosa
Pred: setosa
...
```

Comprendre le script avant de le lancer

```python
# scripts/generate-drift.py (logique essentielle)
NORMAL = {
  "sepal_length": (4.3, 7.9),
  "sepal_width":  (2.0, 4.4),
  "petal_length": (1.0, 6.9),
  "petal_width":  (0.1, 2.5),
}

def sample(drifted=False):
    return [
        round(
          random.uniform(lo, hi)
          * (random.uniform(1.5, 2.5) if drifted else 1.0),
          2
        )
        for lo, hi in NORMAL.values()
    ]
```

Données normales

[5.1, 3.5, 1.4, 0.2]

dans la distribution d'entraînement

Données driftées (facteur ×1.5–2.5)

[9.3, 7.1, 11.2, 0.8]

hors distribution — réponse HTTP 200 quand même

Simuler un drift de données

```
# Exécuter le script de simulation de drift
python scripts/generate-drift.py \
  --endpoint "$SCORING_URL" \
  --n_normal 50 \
  --n_drifted 20

# Ce que le script fait :
# → 50 requêtes normales (valeurs dans la distribution)
# → 20 requêtes driftées (valeurs ×1.5 à ×2.5)
# → l'API répond 200 OK pour TOUTES
# → drift invisible pour une alerte HTTP classique
```

Observation clé : HTTP 200 ≠ prédiction fiable. Le drift de données n'est pas détectable par une alerte sur les failed requests.

Observer dans App Insights

Dans Azure Portal → App Insights → Logs :

```
// Comparer trafic normal vs drifté
requests
| where timestamp > ago(1h)
| summarize count() by bin(timestamp, 5m), success
| order by timestamp asc

// Voir les traces avec les valeurs de features
traces
| where timestamp > ago(1h)
| where message contains "features"
| project timestamp, message
| order by timestamp desc
| take 30
```

Ce qu'on cherche :

RBAC et Key Vault

```
# Afficher / créer le groupe mlops-team
az ad group show --group "mlops-team" --query id -o tsv
# ou créer un groupe de test :
az ad group create --display-name "mlops-team" \
  --mail-nickname "mlops-team"

# Assigner les rôles minimaux
GROUP_ID=$(az ad group show --group "mlops-team" \
  --query id -o tsv)
bash scripts/setup-rbac.sh dev "$GROUP_ID"

# Vérifier les rôles assignés
az role assignment list \
  --assignee "$GROUP_ID" --all \
  --query "[].{role:roleDefinitionName,scope:scope}" \
  -o table

# Tester Key Vault
KV="kv-mlopslab-dev"
az keyvault secret set --vault-name $KV \
  --name "test-secret" --value "secret-key-123"
az keyvault secret show --vault-name $KV \
```

RÉCAP DÉMO 2

Ce qu'on vient de voir

SERVING

- Managed Endpoint déployé ✓
- iris-classifier dans le registre ✓
- AKS vs AML : différence claire ✓

MONITORING

- Alerte Azure Monitor créée ✓
- Trafic normal dans App Insights ✓
- Drift simulé → HTTP 200 ✓
- KQL : traces vs requests ✓

GOUVERNANCE

- RBAC minimal configuré ✓
- Secret lu depuis Key Vault ✓
- 0 secret en clair dans le code ✓

LIMITES DE LA DÉMO

Ce que cette simulation ne couvre pas

Ce qu'on simule ici

- ✅ **Data drift** : features hors distribution
- ✅ Invisible pour une alerte HTTP
- ✅ Détectable via KQL sur les traces

Ce qu'on ne simule pas

- ❌ **Concept drift** : la relation feature→label change
- ❌ Comparaison prédictions vs ground truth
- ❌ Seuil statistique sur distribution (ex: PSI, KL divergence)
- ❌ Retraining automatique déclenché par dérive

Pour aller plus loin : **Azure ML Data Monitor** calcule des métriques de drift automatiquement sur les datasets. Ce n'est pas configuré dans ce lab — c'est une étape suivante naturelle.

PARTIE 5 — GOUVERNANCE

Gouvernance

RBAC, Key Vault et séparation dev/prod

Ce qui rend une plateforme ML crédible en entreprise

RBAC — Moindre privilège

- `AzureML Data Scientist` → scope RG
- `AKS Cluster User Role` → scope AKS seulement
- `Key Vault Secrets User` → scope KV seulement
- Pipeline CI/CD : `Contributor` scope RG, **pas subscription**

Key Vault — Centralisation des secrets

- Secrets API, connexions DB, clés
- Rotation possible sans changer le code
- Audit trail intégré
- Jamais de secret en clair dans le code

Séparation dev / prod

- Resource groups séparés
- GitHub Environment `production`
- Approbation manuelle obligatoire
- Promotion d'artefact (pas rebuild)

ANTI-PATTERNS À CONNAÎTRE

Ce qu'il ne faut pas faire

Anti-patterns CI/CD ML

- Reconstruire différemment en prod
- Promouvoir sans validation ni observation des métriques
- Valider seulement la CI locale
- Ignorer le quality gate ("on verra en prod")

Anti-patterns monitoring

- Confondre "monitoring HTTP" et "monitoring ML"
- Croire qu'un endpoint 200 suffit
- Parler de gouvernance seulement à la fin
- Monitorer la latence mais pas les prédictions

La séparation **identité technique du pipeline** vs **droits donnés à une équipe humaine** est essentielle pour l'auditabilité.

POUR ALLER PLUS LOIN

Ce que ce repo ne montre pas encore

MONITORING ML AVANCÉ

- Suivi statistique de la distribution des features
- Comparaison prédictions vs ground truth
- Détection automatique de concept drift
- Retraining déclenché par seuil

CI/CD AVANCÉ

- Blue/green deployment sur AKS
- Canary releases sur Managed Endpoint
- Tests d'intégration post-déploiement
- Rollback automatique sur dégradation

GOUVERNANCE AVANCÉE

- Azure Policy pour conformité
- Workload Identity pour les pods AKS
- Gestion des coûts et budgets
- Diagnostic settings centralisés

6 idées à emporter

1. La CI ML valide le code et l'exécution dans Azure ML — pas seulement le lint et les tests locaux

2. Le quality gate sur les métriques bloque automatiquement le déploiement si le modèle est insuffisant

3. OIDC = aucun secret longue durée dans GitHub — l'identité du pipeline compte autant que son code

4. AKS et AML Managed Endpoint ne jouent pas le même rôle — le choix dépend de l'organisation, pas de la "pureté technique"

5. HTTP 200 ≠ modèle correct — le drift métier est invisible pour une alerte sur les failed requests

6. La gouvernance (RBAC, KV, séparation dev/prod) n'est pas un "détail enterprise" — c'est ce qui rend une démo reproductible en